INFRASTRUCTURE PORTFOLIO

HomeLab

DevOps Infrastructure & AI/ML Experimentation Platform

- 3-Node Proxmox Cluster
- HashiCorp Nomad Orchestration
- Hybrid AI/ML Infrastructure

Michel Mazza • December 2025

PROJECT OVERVIEW

MISSION

Build a production-grade infrastructure platform for learning modern DevOps practices, container orchestration, and AI/ML experimentation through hands-on implementation.

APPROACH

- Systematic, phase-based development
- Operational maturity before automation
- Documentation-first methodology

FOCUS AREAS

- Container orchestration with Nomad
- Service discovery and observability
- AI/ML workload deployment

VALUES

- Practical over perfect
- Learn by doing
- Incremental progress

Foundation for continuous learning and experimentation

INFRASTRUCTURE ARCHITECTURE

COMPUTE LAYER

3x Proxmox Hosts
Intel-based bare metal servers

Virtual Machine Cluster
Flexible workload distribution

Mac Studio (M1 Max)
GPU-accelerated AI inference

ORCHESTRATION

HashiCorp Nomad
Container orchestration

Consul
Service discovery

Traefik
Reverse proxy

STORAGE

Synology NAS (10TB)

NFS Shared Volumes

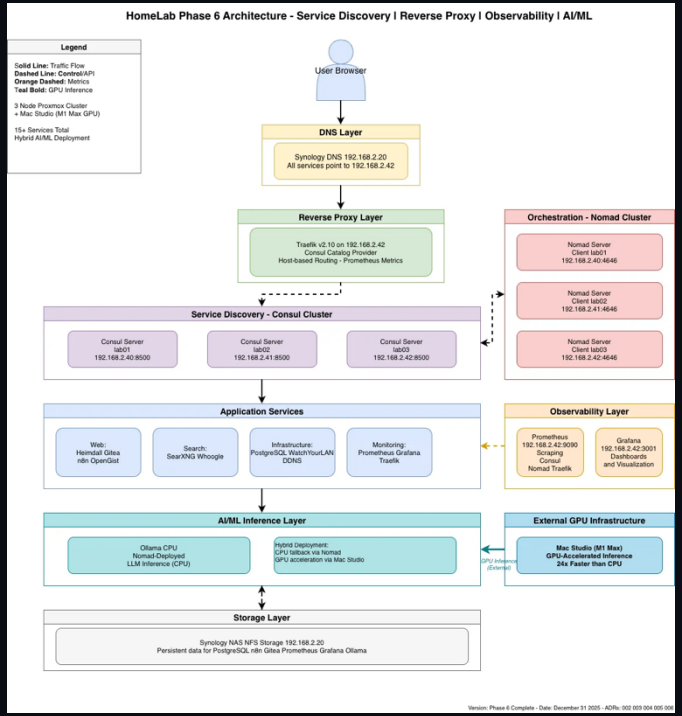
OBSERVABILITY

Prometheus + Grafana

Pushover Alerting

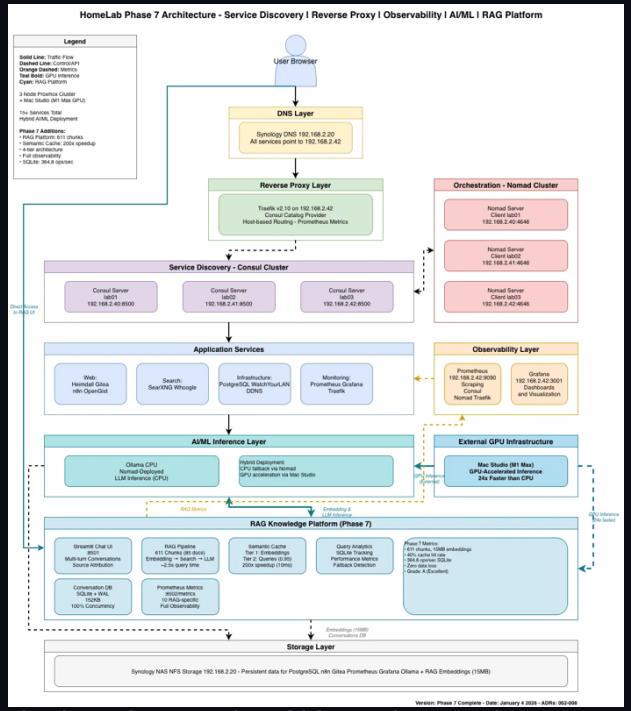
Designed for operational simplicity and learning value

SYSTEM ARCHITECTURE



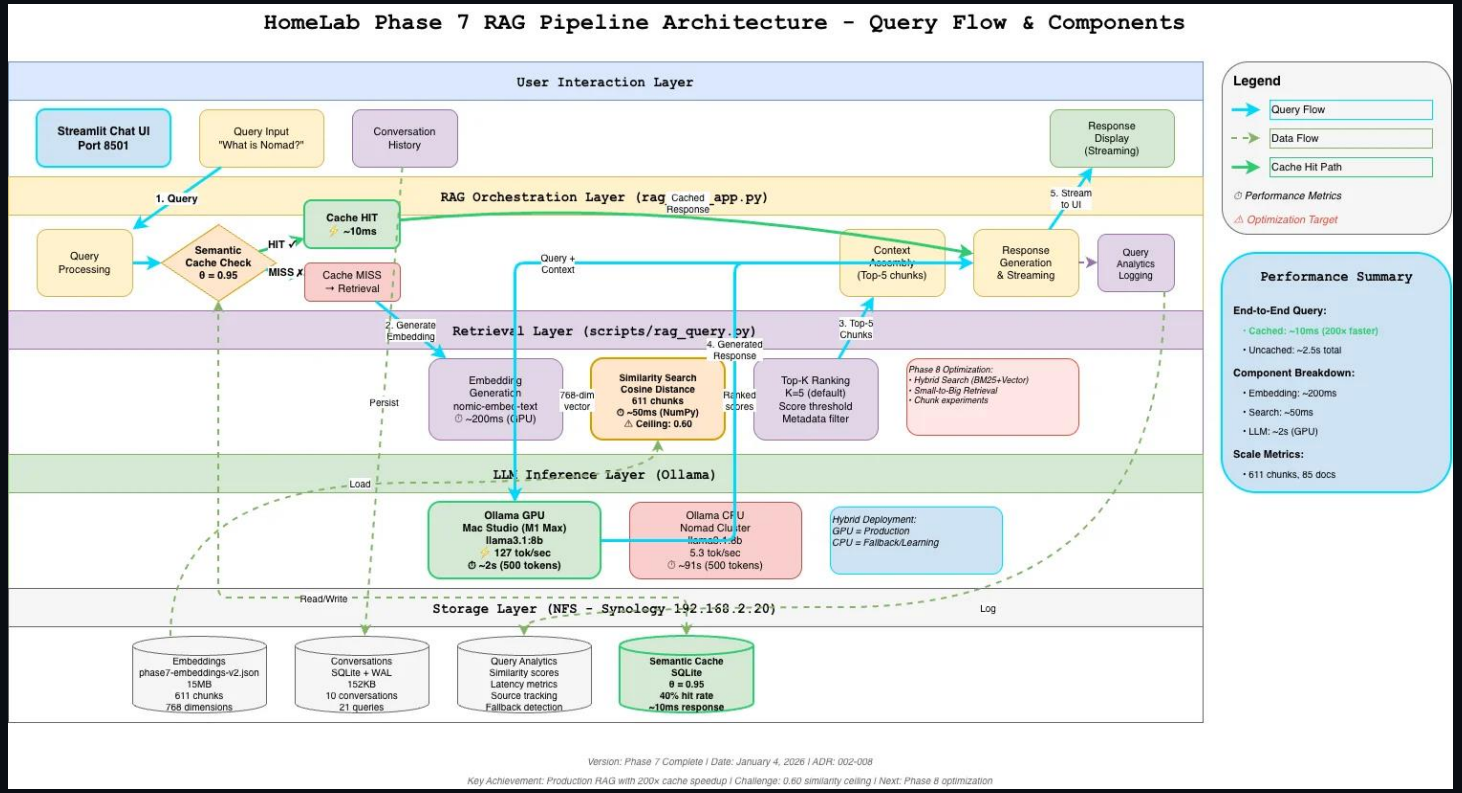
Layered architecture: DNS → Proxy → Service Discovery → Orchestration → Services → AI/ML → Storage

RAG PLATFORM ARCHITECTURE



Phase 7: Production RAG platform - 611 chunks, 85 docs, 2.5s queries, 200× cache speedup

RAG PIPELINE FLOW



5-layer architecture: User → Orchestration → Retrieval → LLM → Storage (2 paths: 10ms cached, 2.5s full)

IMPLEMENTATION JOURNEY

PHASE 1-2 ✓

FOUNDATION

- Hardware inventory documented
- Nomad cluster established
- Architecture decisions recorded

PHASE 3-4 ✓

CORE SERVICES

- Consul service discovery
- Traefik reverse proxy
- Container deployment patterns

PHASE 5 ✓

OBSERVABILITY

- Prometheus metrics stack
- Grafana dashboards
- Mobile alerting system

PHASE 6 ✓

AI/ML PLATFORM

- Hybrid Ollama deployment
- GPU-accelerated inference
- Orchestrated CPU fallback

Progressive buildout with validation at each milestone

THE HYBRID DECISION

Architecture Question: Where Should Ollama Run?

Core Principle

"Learn by doing (CPU), perform in production (GPU)"

Learning Achieved

Nomad AI orchestration mastered

Performance Delivered

24x speedup when needed

Resilience Bonus

Built-in redundancy

Decision: Both endpoints operational, choose based on use case

KEY LEARNINGS

OPERATIONAL MATURITY FIRST

Automate the running system before automating infrastructure provisioning. Manual operations reveal pain points and inform better automation decisions.

PARALLEL ACCESS

Maintain dual access paths during migrations to ensure service availability and enable safe rollback.

HYBRID ARCHITECTURES

Balance learning value with performance by combining CPU and GPU resources strategically.

DOCUMENTATION TIMING

Document decisions while context is fresh rather than retroactively to capture reasoning and alternatives.

FOCUSED SESSIONS

Systematic session-based work produces more reliable results than marathon implementation efforts.

Principles validated through hands-on experience

KEY LEARNINGS

OPERATIONAL MATURITY FIRST

Automate the running system before automating infrastructure provisioning. Manual operations reveal pain points and inform better automation decisions.

PARALLEL ACCESS

Maintain dual access paths during migrations to ensure service availability and enable safe rollback.

HYBRID ARCHITECTURES

Balance learning value with performance by combining CPU and GPU resources strategically.

DOCUMENTATION TIMING

Document decisions while context is fresh rather than retroactively to capture reasoning and alternatives.

RAG IS A SYSTEM

Not just embed + search + LLM. Quality requires measurement, debugging tools are infrastructure.

Principles validated through hands-on experience across Phases 4-7

DEPLOYED SERVICES

ORCHESTRATION

- Nomad (3 servers, N workers)
- Consul service mesh
- Traefik reverse proxy

MONITORING

- Prometheus metrics
- Grafana visualization
- Pushover alerting

AI/ML

- Ollama (Mac Studio GPU)
- Ollama (Nomad CPU)
- Local LLM inference

STORAGE

- NFS mounts from Synology
- Persistent volume support
- Shared data access

APPLICATIONS

- Containerized workloads
- LinuxServer.io images
- Custom deployments

NETWORKING

- Service-to-service discovery
- Automated routing
- Health-based failover

All services registered in Consul with comprehensive health checks and observability coverage

Production-ready services with automated management

THE RETRIEVAL CEILING

THE DISCOVERY

Query "What is Nomad?" returned wrong documents (session notes vs ADR-001 definition). Similarity scores plateau at 0.60 instead of target 0.80+. Systematic investigation: 130 minutes, 4 hypotheses tested, root cause identified as 186-char chunks too small.

This isn't a failure—it's successful engineering. We built a system, understood its limitations through data, and created the tools and framework to improve it systematically.

PHASE 8 PLAN

Three optimization pillars: Hybrid Search (BM25 + Vector), Small-to-Big Retrieval (precise search with full context), and systematic experiments across chunk sizes and embedding models.

Data-driven debugging methodology: Build, measure, learn, iterate

DEPLOYED SERVICES

ORCHESTRATION

Nomad (3 servers, N workers)

Consul service mesh

Traefik reverse proxy

MONITORING

Prometheus metrics

Grafana visualization

Pushover alerting

AI/ML

Ollama (CPU + GPU hybrid)

llama3.1:8b inference

nomic-embed-text embeddings

APPLICATIONS

Heimdall (dashboard)

PostgreSQL (database)

n8n (automation)

INFRASTRUCTURE

NFS storage (Synology)

DNS (internal resolution)

Blackbox exporter

RAG PLATFORM (Phase 7)

Streamlit Chat UI (8501)

Semantic Cache (200× speedup)

SQLite + WAL (152KB)

15+ services orchestrated across 3-node cluster with hybrid GPU acceleration

All services discoverable via Consul, routed through Traefik, monitored by Prometheus

PRODUCTION RAG METRICS

611	85	2.5s	200×
Chunks Embedded	Documents	Query Time	Cache Speedup

INFRASTRUCTURE

15MB embeddings (768 dimensions)

152KB conversation database

SQLite + WAL mode

PRODUCTION USAGE

21 queries with analytics

10 unique conversations

40% cache hit rate

CORPUS DISTRIBUTION

Phase 4: 30.4% (186 chunks)

Phase 5: 22.4% (137 chunks)

Phase 6: 27.2% (166 chunks)

Phase 7: 18.2% (111 chunks)

5.7× scale from Phase 6 (15 docs) to Phase 7 (85 docs) with maintained performance

FUTURE ROADMAP

NEXT PHASE: RETRIEVAL QUALITY OPTIMIZATION (PHASE 8)

Systematic optimization of RAG retrieval quality through hybrid search, improved chunking, and model experiments based on Phase 7 findings.

WEEK 1: HYBRID SEARCH

BM25 + Vector combination

Keyword + semantic signals

Expected: 30-50% improvement

WEEK 2: SMALL-TO-BIG

Search with small chunks

Retrieve full context

Expected: 40-60% fallback reduction

WEEK 3: EXPERIMENTS

Test chunk sizes (512/768/1024)

Compare models (mxbai, e5, bge)

Measure each combination

SUCCESS CRITERIA

Similarity >0.75 for definitional queries

Fallback rate <10% (from 70%)

ADR-001 in top-3 for "What is Nomad?"

Phase 8: Data-driven optimization through three systematic pillars